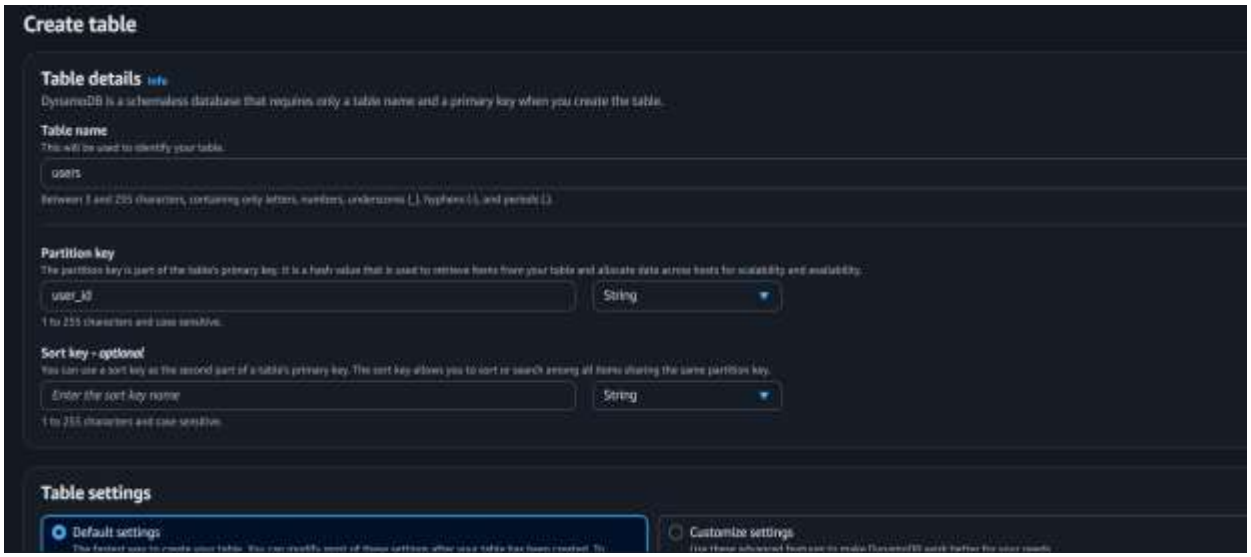
Project: Insert Data into DynamoDB using AWS Lambda

Step 1: Create DynamoDB Table

Go to AWS Console → DynamoDB → Create Table

- Table name: users
- Partition key: user_id (String)

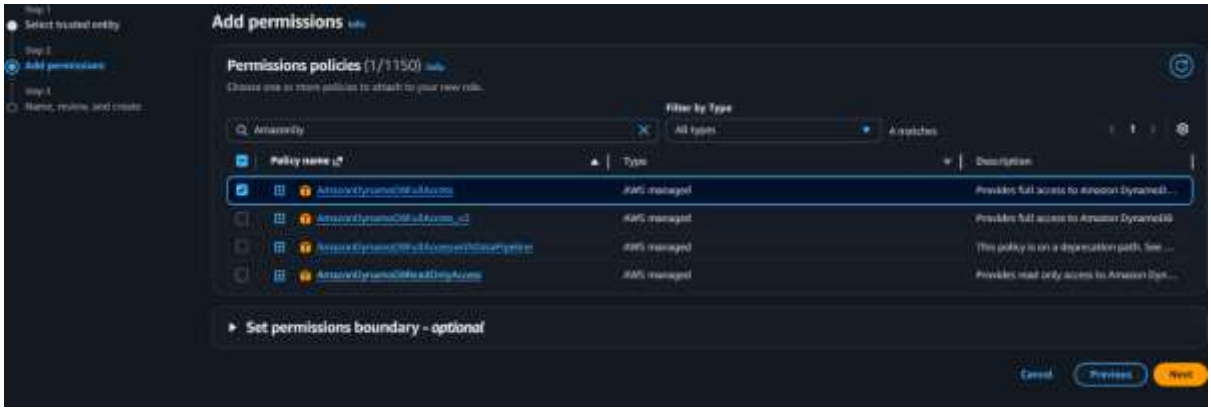


Why this design?

- user_id = unique
- Fast lookup
- No scan needed

Step 2: Create IAM Role for Lambda

Create role with permissions:

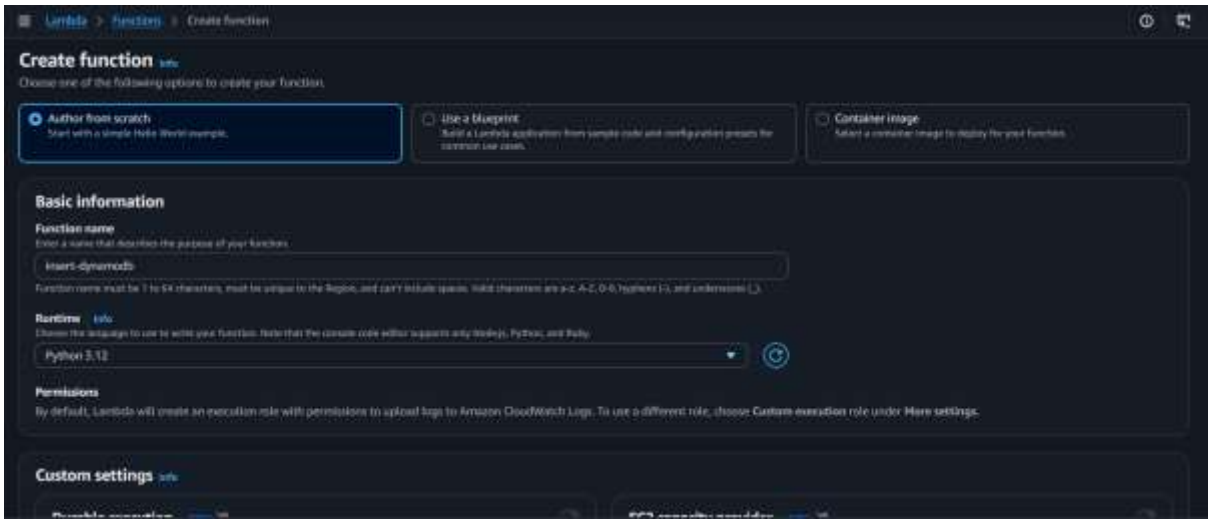


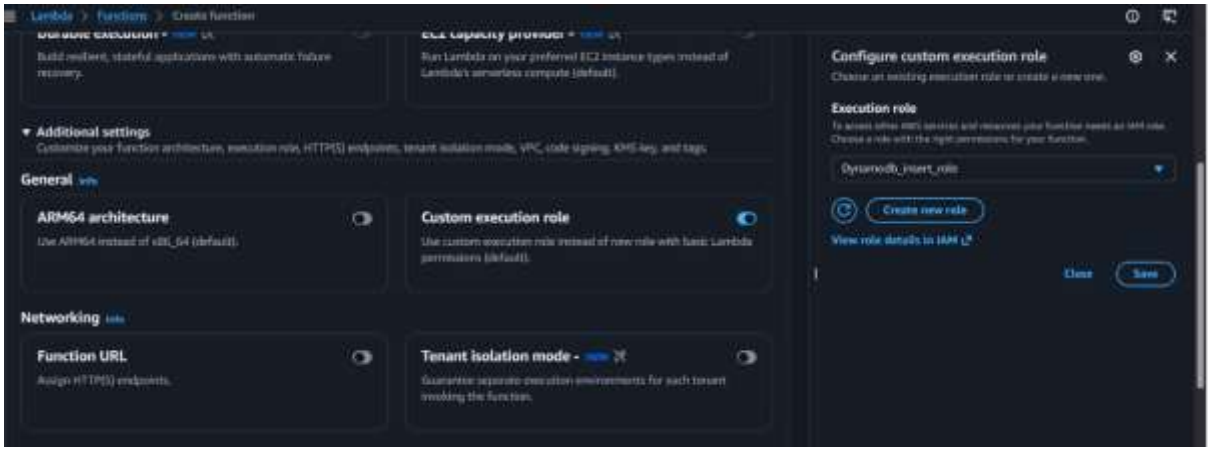
- AmazonDynamoDBFullAccess (for demo only)

In real production → use least privilege

Step 3: Create Lambda Function

- Runtime: Python 3.x
- Attach IAM role

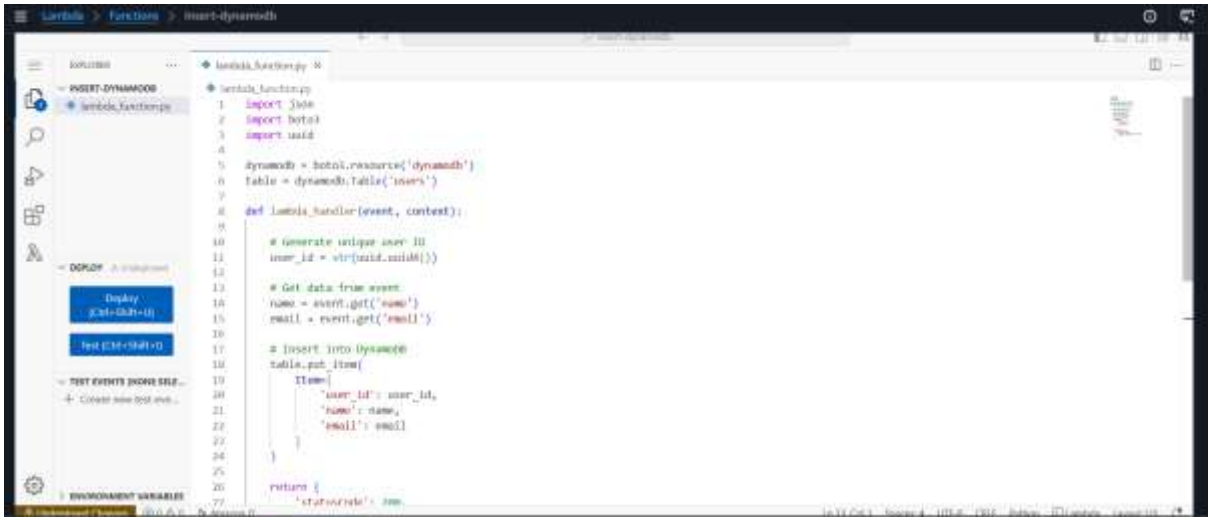




Lambda Code (Core Part)

```
import json

import boto3

import uuid

dynamodb = boto3.resource('dynamodb')

table = dynamodb.Table('users')
```

```
def lambda_handler(event, context):
    # Generate unique user ID
    user_id = str(uuid.uuid4())
    # Get data from event
    name = event.get('name')
    email = event.get('email')
    # Insert into DynamoDB
    table.put_item(
        Item={
            'user_id': user_id,
            'name': name,
            'email': email
        }
    )
    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'User created successfully',
            'user_id': user_id
        })
    }
```



Step 4: Test Lambda

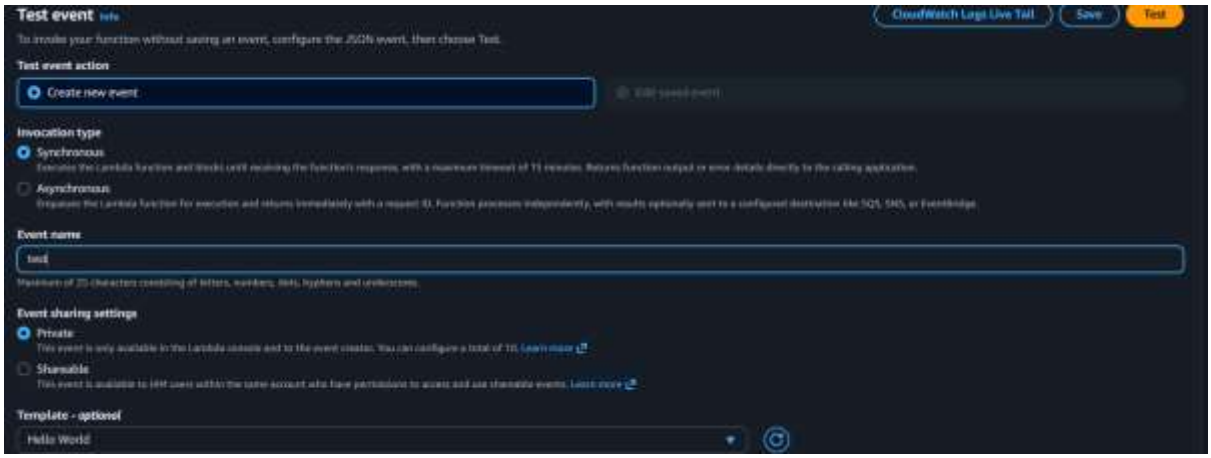
Use test event:

{

"name": "Shravuu",

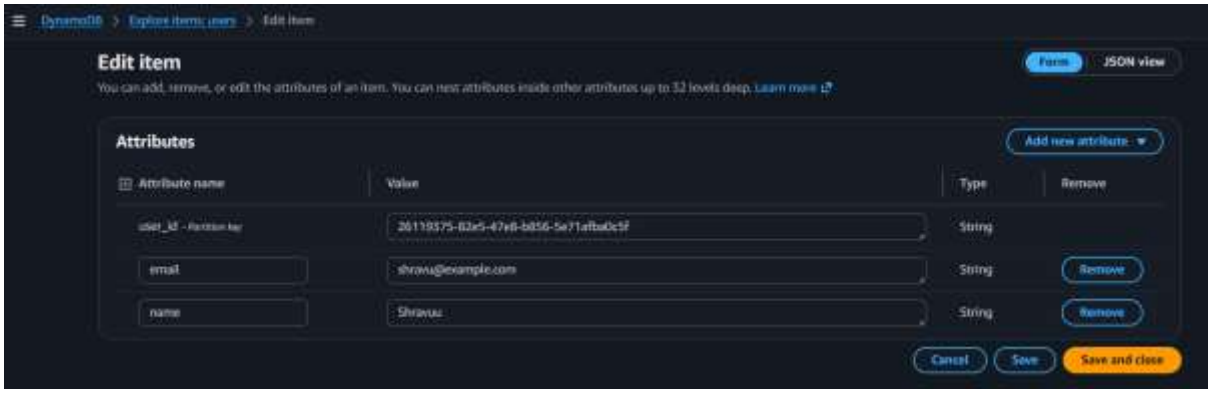
"email": "shravu@example.com"

}



Step 5: Verify in DynamoDB

- Go to table → Explore items
- You should see inserted data

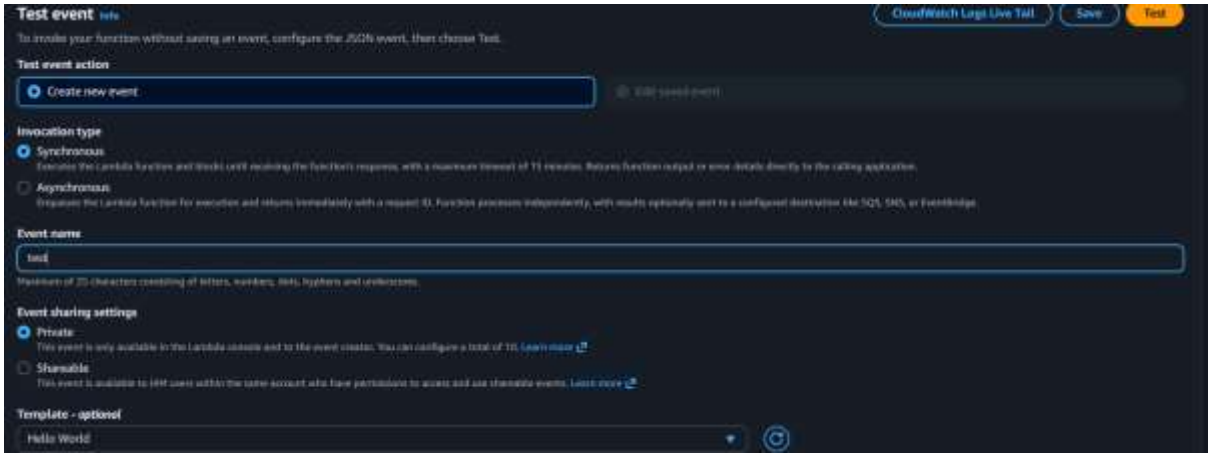


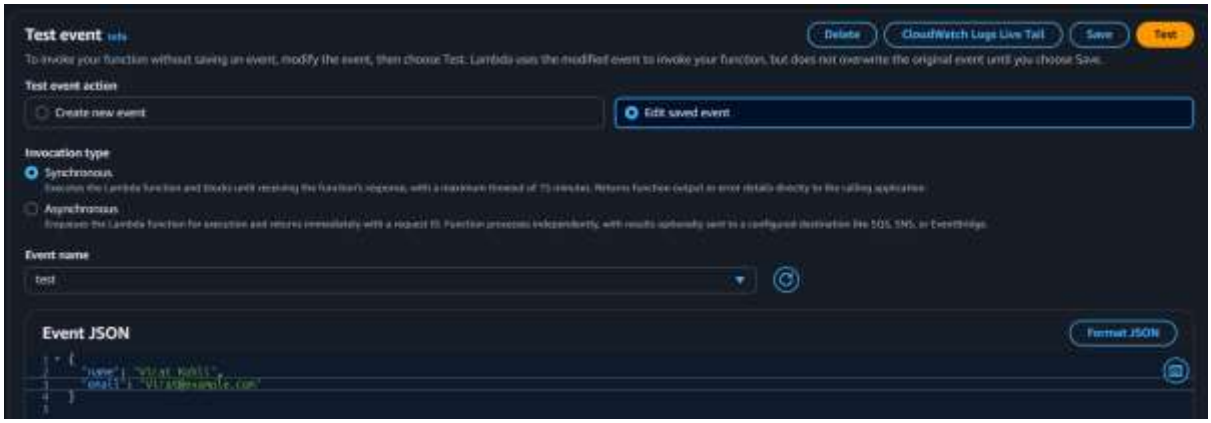
Add Second Data:

Repeat Step 4: Test Lambda:

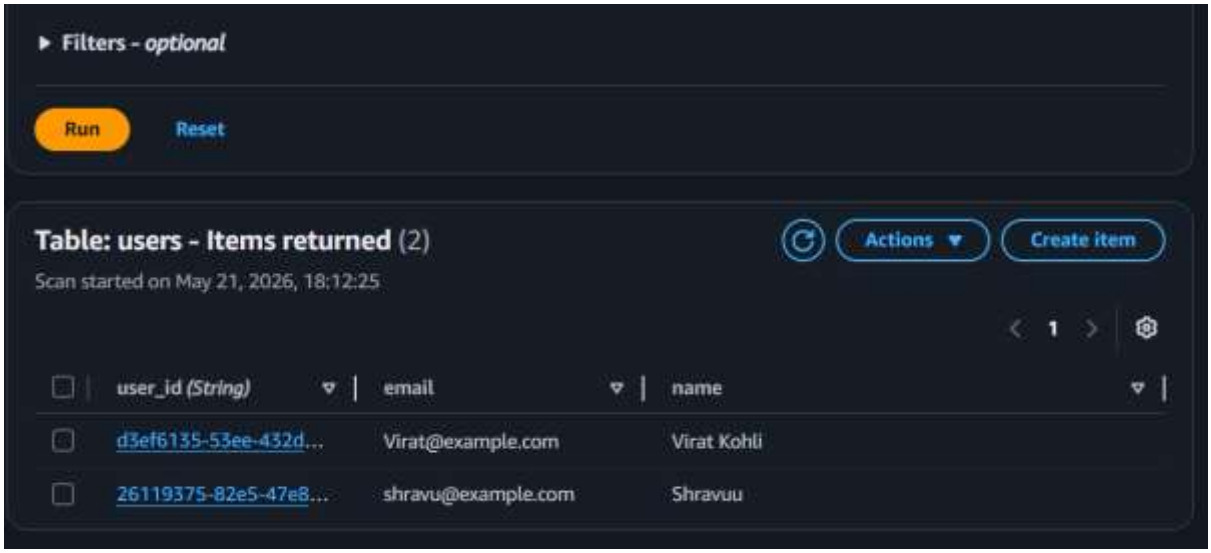
Use test event:

```
{
  "name": "Virat Kohli",
  "email": "Virat@example.com"
}
```





Output:



Learn Here:

- Lambda invocation
- DynamoDB write operation
- IAM role usage
- Event-driven design

UPDATED LAMBDA (Dynamic Table Name + CRUD)

Change Code :

```
import json
import boto3
import uuid
dynamodb = boto3.resource('dynamodb')
# Allowed tables (IMPORTANT for security)
ALLOWED_TABLES = ['users', 'customers']
def lambda_handler(event, context):
    table_name = event.get('table_name')

    # -------- SECURITY CHECK --------
    if table_name not in ALLOWED_TABLES:
        return response("Invalid table name", {})
    table = dynamodb.Table(table_name)
    action = event.get('action')

    # ---------------- CREATE ----------------
    if action == 'create':
        user_id = str(uuid.uuid4())
        name = event.get('name')
        email = event.get('email')
        table.put_item(
            Item={
                'user_id': user_id,
                'name': name,
                'email': email
            }
        )
```

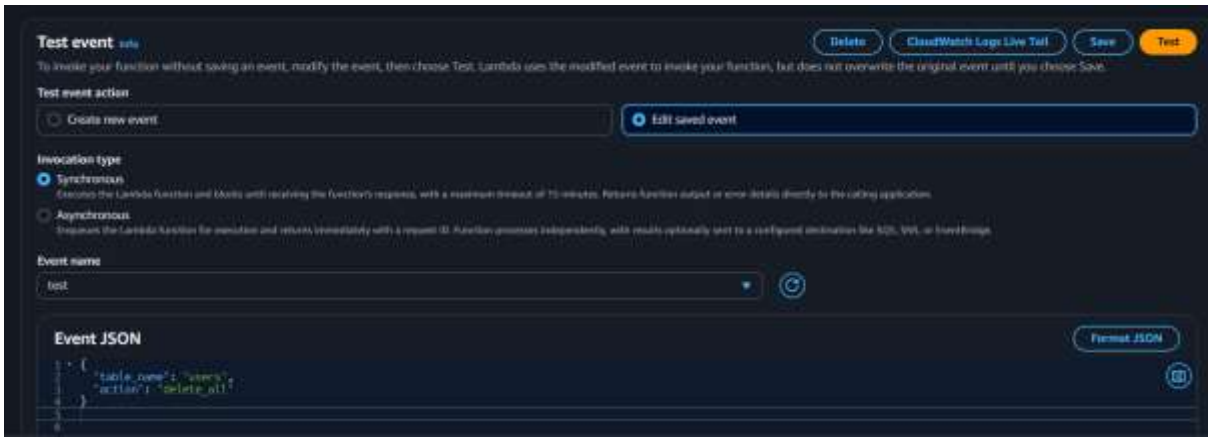
```
    return response("User created", {'user_id': user_id})


# ---------- UPDATE ----------
elif action == 'update':
    user_id = event.get('user_id')
    name = event.get('name')
    email = event.get('email')
    table.update_item(
        Key={'user_id': user_id},
        UpdateExpression="SET #n = :name, email = :email",
        ExpressionAttributeNames={'#n': 'name'},
        ExpressionAttributeValues={
            ':name': name,
            ':email': email
        },
        ReturnValues="UPDATED_NEW"
    )
    return response("User updated", {'user_id': user_id})


# ---------- DELETE SINGLE ----------
elif action == 'delete':
    user_id = event.get('user_id')
    table.delete_item(
        Key={'user_id': user_id}
    )
    return response("User deleted", {'user_id': user_id})


# ---------- DELETE ALL ----------
elif action == 'delete_all':
    scan = table.scan()
```

```
        with table.batch_writer() as batch:
            for item in scan['Items']:
                batch.delete_item(
                    Key={'user_id': item['user_id']}
                )
        return response("All records deleted", {})
    else:
        return response("Invalid action", {})


# -------- Helper --------
def response(message, data):
    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': message,
            'data': data
        })
    }
```

Test Lambda:

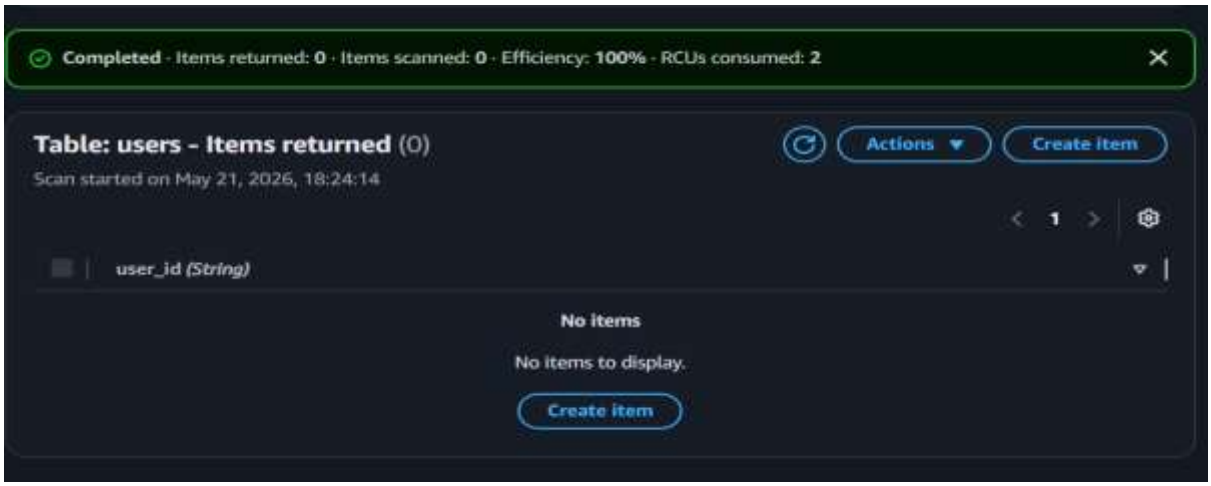
Delete All:

```
{
  "table_name": "users",
  "action": "delete_all"
}
```



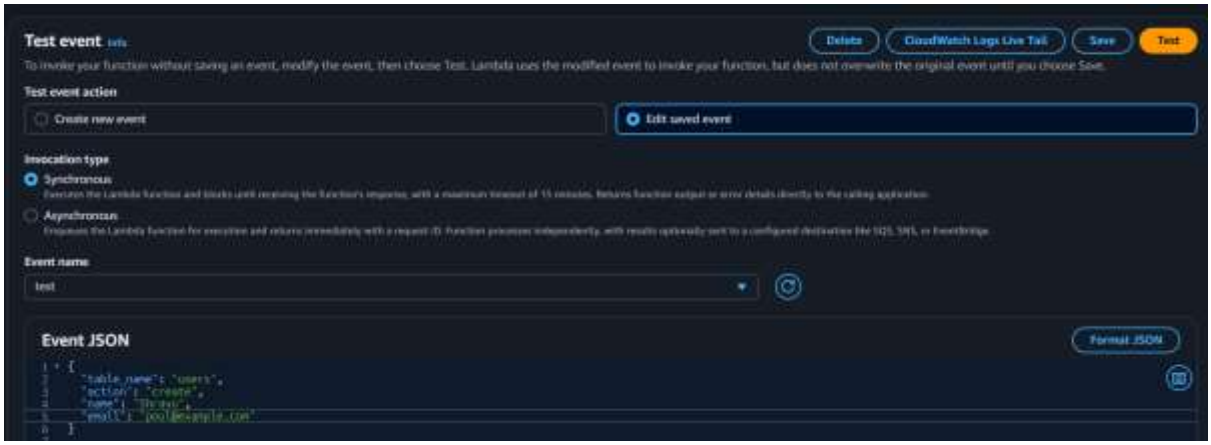
Output:

Delete All Files:



Create:

```
{
 "table_name": "users",
 "action": "create",
 "name": "Shravu",
 "email": "poul@example.com"
}
```



Output:

Create File: